

Agentic Synthesis against Counterexample-Supplemented Sketches

Muness Castle

Eric Rubeck

July 1, 2026

Abstract

Agentic synthesis against counterexample-supplemented sketches is a method for programming with agents under checkable rules: fix a partial program (a *sketch*), maintain a growing observation set E that *supplements* the sketch with counterexamples, and refuse “done” until a gate passes every case.

When validation fails, the failure is promoted into E ; an agent in an IDE-shaped environment edits the sketch and the code and prompts that implement it until gates go green. Solar-Lezama’s sketching and CEGIS [3, 2] supply lineage; our contribution is the closed loop that holds agentic coding accountable to sketch + E —not chat history or spot checks. Hole-filling splits across agent-maintained code (Oracle A) and LLM completion of sketch-declared narrative holes (Oracle B), with hybrid escalation and layered replay/compare validation.

We evaluate the method on a public reference implementation (synthetic roster remediation, 14 scenarios, three gate modes, one recorded promotion session). The domain is illustrative; the technique is domain-agnostic.

1 Introduction

Enterprise remediation is not a chat problem. Policy lives in prose, dates, duplicate clusters, and wire mutations that must close an observed gap. Unconstrained agents paraphrase intent and miss field-level invariants; pure rules lag the playbook SMEs edit.

Agentic synthesis against counterexample-supplemented sketches fixes a **constraint environment** around the agent:

1. A **sketch** S —partial program: fixed operations, documented holes.
2. A **counterexample-supplemented** corpus E —observations promoted when validation fails.
3. A **gate**—replay plus golden compare over all of E ; pass/fail evidence, not narrative confidence.
4. **Agentic synthesis**—the developer or coding agent edits S , Oracle A (encodable holes in code), and Oracle B (sketch-declared LLM holes in prompts) until all target gates pass.

This is counterexample-guided inductive synthesis (CEGIS) [2] with the sketch as stable contract and E as the growing boundary of “done.” Solar-Lezama’s Sketch [3] assumed one synthesizer searching holes; in agentic practice, holes split across *code the agent writes* and *LLM calls the sketch declares*, with hybrid escalation when code abstains or replay rejects a row.

Sect. 2 defines the constraint environment generically. Sects. 3–7 instantiate sketch semantics and validation on a **worked example** (workforce roster remediation) so readers can reproduce gates without client data. Sect. 10 walks one counterexample promotion on that example.

1.1 Relation to algorithmic cegis

Solar-Lezama’s lineage splits into *automated* and *agentic* poles that share loop grammar but not synthesizers. Do not collapse them—they fail differently.

	Sketch compiler [3]	Z3 CEGIS [4]	Agentic CE-supplemented sketches (this work)
Sketch	Sketch language + holes	SMT impl / input / helper vars	Prose + partial program file
Synthesizer	Built-in search	<code>findImplementation</code> (SMT)	Agent edits Oracle A / Oracle B
Counterexample	Validator search	<code>findCounterExample</code> (SMT)	Gate notes $\rightarrow$ promote into E
Validation	Sketch semantics	Correctness constraints	Replay + golden compare on E
Coverage	$\forall$ inputs (via CE growth)	Minimal input subset	Bounded observation on E

Classic CEGIS libraries such as `marcelwa/CEGIS` automate alternation between implementation search and counterexample discovery inside an SMT solver. Our harness keeps the **validation discipline** of that loop but replaces automated synthesis with **agent-performed** edits to sketch, code, and prompts—unlike `CEGISHandler::CEGISRoutine()`, which needs no IDE-shaped agent.

2 The constraint environment

2.1 Artifacts the agent may edit

The agent does not patch production payloads ad hoc. It edits auditable artifacts that the gate evaluates:

Artifact	Role
Sketch $\mathcal{S}$	Partial program; SME- and agent-editable
Oracle A	Code implementing encodable sketch clauses
Oracle B prompts	LLM path for sketch-declared non-code holes
Corpus E	Observations; failures promote new counterexamples
Oracle B cassettes	Frozen LLM output for CI (optional)

2.2 What the gate runs

Each gate invocation over E :

1. **Resolve** rows (deterministic, hybrid, or LLM-only mode).
2. **Replay** — does the suggestion close the domain gap?
3. **Compare** — does it match golden fields replay under-specifies?
4. **Report** per-scenario pass/fail.

A prompt-dump command (e.g. `bench prompt` in our reference implementation) exposes live Oracle B system and user text so prompt edits stay inspectable.

2.3 Agentic coding inside the loop

Run gate $\rightarrow$ read failure notes $\rightarrow$ edit sketch / Oracle A / Oracle B $\rightarrow$ rerun until green.

Oracle A is **code the agent maintains** under gate pressure—not a black-box synthesizer. Oracle B completes sketch-declared narrative holes. Hybrid keeps A on the hot path and escalates to B on abstention or replay failure (Sect. 5).

2.4 Counterexample-supplemented sketches

Corpus E is not a static test suite. It **grows when validation fails**: a gate note such as **expected op modify**, **got append** promotes a scenario, tightens S , and drives edits to Oracle A or Oracle B. Convergence means all target gates pass on the current E —permission to ship regression, not proof over all inputs (bounded observation hypothesis [3]).

2.5 Reference implementation (worked example)

To make the method runnable, we ship a public reference implementation on a **synthetic workforce roster** domain (badge hours vs. scheduled hours). Repository name *RosterSynth*; illustrative only. Artifact paths: `docs/sketch.md`, `playbook.py`, `oracle/prompt.py`, `scenarios/*.json`, `bench gate`, `bench oracle`, `bench prompt`. Corpus `reference-v1`: 14 scenarios, cassette-backed CI, session transcripts. Sections 3–8 use this example; the method transfers to any sketch with replay, compare, and a scenario corpus.

3 Sketch language (worked example)

The following instantiates S on the roster reference implementation. Other domains replace wire format and hole rules; the CE-supplemented agent loop is unchanged.

The sketch extends a fixed wire format with prose hole rules. A suggestion is either an append adjustment or a modify that sets `status=4` on one booking.

3.1 Sketching with adjustment holes (Op 1)

When no duplicate cancellation closes the gap, the sketch fixes `op=append` and leaves `shiftKind`, `hours` (typically Δ), and `workDate` open. Work-date policy: use pay-window end w when the latest active booking falls in $[w^-, w)$; else `max(workDate)` among active bookings; else w .

3.2 Higher-level duplicate holes (Op 2)

When twin active bookings on w share (`shiftKind`, `hours`), the sketch fixes `op=modify` and `fields.status=4`. The synthesizer chooses `bookingId`. Default tie-break: the higher id, only if canceling that booking alone closes Δ .

3.3 Abstraction in the sketch (Op 2b)

Some payloads carry `clusterNotes`: payroll text on an ambiguous duplicate cluster. The sketch marks these as deliberate non-encoding: Oracle A must abstain. Oracle B reads tags, narrative, and sketch prose. Matching scenarios set `requiresLlmFallback`.

3.4 Putting it all together

Oracle A and Oracle B both emit one row per imbalanced employee. The sketch picks the operation key; implementations fill holes; Sect. 7 decides acceptance.

4 Semantics of sketches

Each employee state σ has badge hours B , scheduled rollup R , window w , bookings, and optional `clusterNotes`. Badge hours are never edited. Define

$$\Delta = B - R. \quad (1)$$

Applying suggestion ϕ yields effective gap Δ_ϕ under replay rules in `verifier.py`. Row ϕ is *replay-valid* when $|\Delta_\phi| < \epsilon$.

Golden expectation g adds fields replay under-specifies. Row ϕ is *compare-valid* for g when `comparer.py` matches `op`, `workDate`, `bookingId`, and related columns.

A row is *valid* on (σ, g) when it is both replay-valid and compare-valid. Replay alone is not enough: an append can close Δ while using the wrong operation—compare exists because we initially treated “delta closed” as “done.” That failure mode motivated Op 2 in Sect. 10.

5 Resolution in the harness

Let $E = \{(\sigma_i, g_i)\}_{i=1}^n$ be observations from `scenarios/*.json`. Hybrid strategy $\mathcal{H} = (A, B, \text{hybrid})$ is *valid on E* when, for every $(\sigma, g) \in E$, hybrid resolution on σ yields a row valid on (σ, g) .

The agent’s job in the harness is to refine $\mathcal{S}$, Oracle A, and Oracle B sketch alignment until $\mathcal{H}$ is valid on E . Hybrid resolution per employee: $r_A \leftarrow A(\sigma)$; if r_A is missing or replay-invalid, $r \leftarrow B(\sigma)$; else $r \leftarrow r_A$.

6 The cegis-shaped agent loop

Figure 1 is the harness loop: validation (gates) and repair (agent + oracles) are separate modules tied by counterexample promotion. **Synthesis is agent-performed:** the developer or coding agent edits artifacts between gate runs. This is not automated CEGIS synthesis as in Z3’s `CEGISRoutine()` [4], which alternates SMT solvers without an IDE-shaped editor.

Input: sketch $\mathcal{S}$, corpus E , harness (**bench**)

1. Agent edits $\mathcal{S}$, Oracle A code, Oracle B prompts
2. Synthesize: run **bench gate** (deterministic / hybrid / llm-only)
3. Validate: replay + compare per scenario in E
4. If all pass: done
5. Else: promote failure notes to E ; goto 1

Figure 1: CEGIS-shaped agent loop in a counterexample-supplemented sketch harness.

Oracle A implements `playbook.build_rows`. Oracle B calls `Bedrock Converse` with tool `emit_roster_suggestions` [1] or reads `cassettes/*.json` in CI.

bench gate `--mode  $m$` walks the manifest, resolves rows under m , runs verifier and comparer, and aggregates results. Mode `deterministic` skips scenarios tagged `requiresLlmFallback` so Oracle A is scored on the rule-encodable subset (12 of 14 in corpus `reference-v1`).

7 Validation

Validation is layered so the harness returns actionable failure modes:

Replay (`verifier.py`): appends count toward closing Δ only when `workDate` $\leq w$; cancels remove scheduled hours from active bookings on or before w .

Golden compare (`comparer.py`): checks promoted policy fields replay under-specifies. Compare caught kiosk append rows that replay closed but used the wrong `op`—the signal that replay-only validation would have shipped the wrong fix.

8 Experience on the reference implementation

All runs below use the roster worked example, cassette Oracle B, no AWS—matching CI.

Gate	Result	Role in harness
Deterministic	12/12	Agent-written Oracle A on encodable subset
Hybrid + cassette	14/14	Production-shaped path
LLM-only + cassette	14/14	Oracle B ablation

Corpus `reference-v1` has 14 scenarios. Gates finish in seconds on a laptop. The repository includes `bench gate|oracle|prompt`, session docs with command transcripts, and `scripts/record_session.sh`.

9 Comparison with unconstrained agents and automated cegis

Versus unconstrained agentic coding. Without the harness, an agent edits code until tests “look right”—often replay-only checks or spot checks. The reference implementation forces three separable accountabilities: sketch clauses, Oracle A implementation, and Oracle B prompt/cassette alignment, each falsifiable by gate mode.

A single Oracle B on every row merges failure modes (sketch lag vs. prompt drift vs. narrative holes). Hybrid plus sketch-declared abstention keeps agent-written code on encodable policy while the LLM handles prose the sketch marks non-encoding.

Versus automated cegis (Z3). SMT CEGIS [4] searches implementations and discovers counterexamples inside the solver; correctness is encoded as constraints over `impl`, `input`, and helper variables. That path requires encodable policy and does not accommodate sketch-declared narrative holes (Sect. 3, Op 2b). Our method inherits CEGIS loop structure—grow E , refuse done until gates pass—but locates synthesis in the agent and locates validation in replay/compare over a promoted scenario corpus rather than in a second SMT solver. The reference implementation is a **constraint environment for agentic coding**, not a drop-in replacement for `marcelwa/CEGIS`.

10 Worked example: kiosk counterexample promotion

One promotion on the roster reference implementation, with commands and output in `docs/sessions/01-kiosk-d`

Counterexample in E . Employee E-1800: $B = 40$, $R = 80$, $\Delta = -40$, twin 40h bookings on w . Golden expects `modify` on booking 1802.

Gate failure (historical). With only Op 1 in $\mathcal{S}$, Oracle A appends `−40h`. Replay passes; compare fails (expected `op modify`, got `append`). The harness note tells the agent to extend the sketch, not tweak the LLM.

Agent edits sketch + Oracle A. Add Op 2 to $\mathcal{S}$; implement `_try_cancel_duplicate`. Deterministic gate passes this scenario.

Agent aligns Oracle B. Wrong cassette (lower `bookingId`) fails `llm-only` compare until sketch prose, `prompt.py` decision order, and cassette align. `bench prompt` dumps the live prompts for inspection.

Sketch-declared LLM holes. `cluster_notes_*` scenarios: Oracle A abstains; hybrid requires Oracle B. Deterministic gate excludes them (`requiresLlmFallback`).

11 Conclusions

We demonstrate **agentic synthesis against counterexample-supplemented sketches**: sketch + growing E , IDE-shaped gates, agent-edited Oracle A and Oracle B until pass. Dual oracles and hybrid escalation are mechanisms inside that loop. The roster domain is one worked example; the method transfers wherever sketch, replay, compare, and a scenario corpus apply. Run `bench gate` on cassettes to reproduce the experience table without AWS.

Open work: richer repair loops, multi-row fixes per employee, and tracking live-Bedrock drift when the sketch changes.

References

- [1] Amazon Web Services. Carry out a conversation with messages using the `converse api` operations. <https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html>, 2024.
- [2] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008.
- [3] Armando Solar-Lezama. Program sketching. *International Journal on Software Tools for Technology Transfer*, 15:475–495, 2013.
- [4] Marcel W. CEGIS: Counter-example guided inductive synthesis for Z3. <https://github.com/marcelwa/CEGIS>, 2016. Open-source CEGIS library for Microsoft Z3.